The whole deployment, in order

START HERE

This guide starts with the files already prepared on your Linux laptop. It takes you through building and checking Restormer + YOLOv8n on your Jetson Orin Nano, then collecting measured results. No DGX commands or retraining are needed for this workflow.

Three machines. Only two are needed now.

LOCAL / LAPTOP — your bms@sky computer. The project is at /home/bms/Desktop/BTP_Aegis-Perception. Use it to send the bundle and retrieve results.

JETSON / ORIN NANO — the 4 GB or 8 GB device that will run inference. Build TensorRT engines and perform device measurements here.

DGX / LOGINNODE — previously used for training. Do not submit Slurm jobs or use the DGX SSH address for the commands in this guide.

What you have → what you will create

Now: validated FP32 ONNX graphs for Restormer (~100.85 MiB) and YOLOv8n (~12.23 MiB). ONNX stores computation and weights; it is not itself an INT8 model.

On Jetson: TensorRT .engine files built from those graphs. The comparison tries FP32, mixed FP16 and calibrated mixed INT8 Restormer, with one fixed FP16 YOLO detector. Sensitive restoration math stays FP32.

Follow this order

Page 2. Copy the latest bundle and identify the Jetson.
Page 3. Prepare the environment and launch the comparison.
Page 4. Read status and run one restored image.
Page 5. Compare raw/restored inputs; optionally use a recorded video.
Page 6. Optionally test the camera, then collect results.
Page 7. Bring results back to the laptop.
Page 8. Reconnect or troubleshoot when needed.

Before starting: obtain the Jetson's SSH username and IP address from its owner. It needs a working JetPack CUDA/TensorRT installation, power, cooling and network access. This guide does not install or flash JetPack.

Copy one command box at a time. Wait for it to finish and read the expected result. Stop at an error. Do not paste terminal prompts such as bms@sky:$ or cs23b051@loginnode:$ into commands.

Status on 24 October 2026: local export/preparation complete; Jetson execution unmeasured. The pruning screen rejected its candidates. The original trained model and Chitraksh's files are unchanged.

Laptop → Jetson

01 / TRANSFER

Open a terminal on your laptop. Enter the Jetson's actual user@IP when prompted. Use the latest compression bundle (~136 MiB), which already includes both ONNX models, calibration inputs and device scripts.

LOCAL — laptop terminal

```
cd /home/bms/Desktop/BTP_Aegis-Perception
BUNDLE=aegis_restormer_edge_compression_20261024
read -r -p "Jetson login (user@IP): " JETSON_TARGET
(cd artifacts/releases && sha256sum -c "$BUNDLE.tar.gz.sha256") &&
ssh "$JETSON_TARGET" 'mkdir -p ~/aegis-edge' &&
scp "artifacts/releases/$BUNDLE.tar.gz" \
  "artifacts/releases/$BUNDLE.tar.gz.sha256" \
  "$JETSON_TARGET:~/aegis-edge/"
```

Expected: checksum OK, then both files transfer. On first connection, verify the SSH host fingerprint with the Jetson owner before accepting it. The requested password is the Jetson account password.

LOCAL — same laptop terminal; this enters the Jetson

```
ssh "$JETSON_TARGET"
```

JETSON — confirm you reached the correct machine

```
hostname
tr -d '\0' < /proc/device-tree/model
printf '\n'
uname -m
free -h
df -h "$HOME"
```

Expect an Orin Nano model name and aarch64. If you see your laptop, loginnode, x86_64, or an original non-Orin Nano, stop and check the host. Disk space and working RAM are different; engine building needs more than the archive size.

JETSON — verify and extract, first time only

```
cd "$HOME/aegis-edge"
BUNDLE=aegis_restormer_edge_compression_20261024
sha256sum -c "$BUNDLE.tar.gz.sha256" &&
test ! -e "$BUNDLE" &&
tar -xzf "$BUNDLE.tar.gz" &&
cd "$BUNDLE"
```

If the extracted directory already exists, the test command stops extraction intentionally. Do not extract over existing results. Use the reconnect block on page 8 for an already verified installation.

Prepare once, then start the comparison

02 / BUILD

You should now be inside the extracted bundle on Jetson. The environment uses JetPack's existing CUDA, TensorRT and OpenCV. It does not require PyTorch on the device. Do not install desktop CUDA/TensorRT packages over JetPack.

JETSON — environment and prerequisite checks

```
python3 -m venv --system-site-packages .venv &&
.venv/bin/python -m pip install \
  -r configs/deployment/edge_runtime_requirements.txt &&
export AEGIS_PYTHON="$PWD/.venv/bin/python" &&
"$AEGIS_PYTHON" scripts/deployment/edge_verify_package.py &&
"$AEGIS_PYTHON" scripts/deployment/edge_preflight.py \
  --output results/preflight_manual.json
```

Expected: all 44 packaged file hashes verify and preflight reports passed: true. The supported TensorRT API range is 8.5/8.6 or 10.x. If a prerequisite fails, use page 8 before launching.

Run this launch block once

JETSON — background build, validation and benchmark

```
mkdir -p results
if [ -e results/compression_driver.pid ]; then
  echo "Launch record exists. Check status before rerunning."
else
  nohup bash -c '
    bash scripts/deployment/edge_compression_device.sh
    run_status=$?
    printf "%s\n" "$run_status" > results/compression_exit_code.txt
    exit "$run_status"
  ' > results/compression_driver.log 2>&1 < /dev/null &
  echo "$!" > results/compression_driver.pid
  cat results/compression_driver.pid
fi
```

A process number is printed. The job builds the fixed FP16 detector, then tries FP32, FP16 and INT8 Restormer sequentially. INT8 uses 64 training-only calibration inputs. Candidates are checked and successful ones benchmarked; failures stay in the logs.

JETSON — watch progress

```
tail -F results/compression_driver.log
```

Ctrl+C here stops only the log viewer. The background comparison continues. nohup is intended to survive an SSH disconnect; it does not survive a Jetson shutdown or reboot. Keep the Jetson powered.

Compilation may be quiet for a while. Do not start a second copy because the log pauses. There is no measured runtime estimate for your device yet. Page 4 shows how to distinguish running from done.

Read the status; try one image

03 / CHECK

Stop watching the log with Ctrl+C, or reconnect using page 8. Run these commands from the extracted bundle directory.

JETSON — is the comparison finished?

```
if [ -f results/compression_exit_code.txt ]; then
  printf 'Finished. Exit code: '
  cat results/compression_exit_code.txt
else
  ps -p "$(cat results/compression_driver.pid)" -o pid,etime,stat,args
fi
```

Exit 0: all three trials completed. **Nonzero:** at least one step failed; read its log. **No exit file, process listed:** still running. **No exit file and no process:** interrupted unexpectedly; preserve logs and investigate.

JETSON — after completion, read each precision's status

```
cat results/*_compression/status.tsv
ls results/*_compression/device_comparison.csv
```

If status.tsv does not exist, preflight or the detector build may have failed first: read compression_driver.log. Rejected INT8 does not invalidate successful FP16. The CSV contains completed measurements, not automatic accuracy approval.

JETSON — choose the completed FP16 candidate for the demo

```
export REST_ENGINE=engines/comparison_restormer_fp16.engine
export DET_ENGINE=engines/comparison_yolov8n_fp16.engine
test -f "$REST_ENGINE" && test -f "$DET_ENGINE" &&
printf 'Engine files present; the runner will verify them.\n'
```

Continue only if FP16 completed and both files exist. If FP16 failed but FP32 completed, run: export REST_ENGINE=engines/comparison_restormer_fp32.engine. If neither passed, stop restoration inference and return the logs. Never select a .candidate.engine or automatically choose INT8 just for speed.

JETSON — one restored image, saved for inspection

```
IMAGE_RUN="results/image_$(date +%Y%m%d_%H%M%S)"
"$AEGIS_PYTHON" scripts/deployment/edge_run.py \
  --restorer "$REST_ENGINE" --detector "$DET_ENGINE" \
  --source samples/fog.jpg --mode restored --warmup 10 \
  --save-visuals --output "$IMAGE_RUN"
```

Expected: 000000.jpg, detections.jsonl and benchmark.json inside the new result directory. The runner verifies engine checksums and validation records before inference.

Raw versus restored, on identical inputs

04 / COMPARE

Keep the chosen engine, detector, power mode and cooling unchanged. This comparison separates restoration from resizing effects. It repeats three bundled images to measure throughput, not detection accuracy.

JETSON — paired sample benchmark

```
PAIR_DIR="results/paired_$(date +%Y%m%d_%H%M%S)"
mkdir -p "$PAIR_DIR"
for VIEW in raw resize_control restored; do
  "$AEGIS_PYTHON" scripts/deployment/edge_run.py \
    --restorer "$REST_ENGINE" --detector "$DET_ENGINE" \
    --source samples --mode "$VIEW" --warmup 20 --repeat 100 \
    --output "$PAIR_DIR/$VIEW" || break
done
"$AEGIS_PYTHON" scripts/deployment/edge_summarize_device.py \
  --results "$PAIR_DIR"
```

Expected: three rows in device_comparison.csv. If an error stops the loop, fewer rows mean an incomplete comparison. These foreground commands need a connected session until completion.

Optional: compare one recorded MP4

Open a second LOCAL laptop terminal for the upload. Keep the Jetson terminal open. Choose one local MP4 and send it as test.mp4.

LOCAL — second laptop terminal

```
read -r -p "Jetson login (user@IP): " JETSON_TARGET
read -r -p "Full local path to an MP4: " VIDEO_FILE
test -f "$VIDEO_FILE" &&
ssh "$JETSON_TARGET" 'mkdir -p ~/aegis-edge/media' &&
scp "$VIDEO_FILE" "$JETSON_TARGET:~/aegis-edge/media/test.mp4"
```

JETSON — back in the existing Jetson terminal

```
VIDEO_RUN="results/video_$(date +%Y%m%d_%H%M%S)"
for VIEW in raw restored; do
  "$AEGIS_PYTHON" scripts/deployment/edge_run.py \
    --restorer "$REST_ENGINE" --detector "$DET_ENGINE" \
    --source "$HOME/aegis-edge/media/test.mp4" --mode "$VIEW" \
    --warmup 10 --max-frames 300 --save-visuals \
    --output "$VIDEO_RUN/$VIEW" || break
done
```

Both runs use the same first 300 frames, or the whole video if shorter. Output annotated.mp4 FPS metadata is not processing FPS; read benchmark.json. Saving video adds I/O cost, so do not compare these timings directly with no-save runs.

Optional camera test, then export results

05 / COLLECT

Optional: USB camera, 300 frames

Connect a USB/V4L2 camera to the Jetson. Camera 0 is the first index; camera 1 may be needed if another device occupies index 0. A CSI ribbon camera may require separate capture setup.

JETSON — list camera device nodes

```
ls /dev/video*
```

JETSON — restored camera demo

```
CAMERA_RUN="results/camera_$(date +%Y%m%d_%H%M%S)"
"$AEGIS_PYTHON" scripts/deployment/edge_run.py \
  --restorer "$REST_ENGINE" --detector "$DET_ENGINE" \
  --source camera:0 --mode restored --max-frames 300 \
  --warmup 10 --save-visuals --output "$CAMERA_RUN"
```

This saves annotated.mp4, detections.jsonl and benchmark.json. It does not open a live GUI window. Keep SSH connected during this foreground demo. For a fair comparison use the same recorded video, not two different live scenes.

Required: collect results after all runs finish

Collect logs and metadata even if a build failed. Engine JSONs include quality checks, sizes and layer details. Large engine binaries stay on Jetson. The archive includes saved videos, so its size depends on the optional demos you ran.

JETSON — create a results archive and checksum

```
mkdir -p results/engine_metadata
for META in engines/*.json; do
  if [ -f "$META" ]; then cp "$META" results/engine_metadata/; fi
done
tar -czf "$HOME/aegis-edge/aegis_jetson_results.tar.gz" results &&
cd "$HOME/aegis-edge" &&
sha256sum aegis_jetson_results.tar.gz \
  > aegis_jetson_results.tar.gz.sha256
```

You are now in ~/aegis-edge on Jetson. Leave the results and engines in place. Page 7 uses a LOCAL laptop terminal to copy the archive back; do not run its local commands inside SSH.

What gets preserved

Build evidence: preflight, build logs, pass/fail status, numerical validation and engine layer information.
Measurements: benchmark JSONs, comparison CSVs and tegrastats logs when available.
Examples: saved image/video overlays and detection coordinates.

Bring the evidence back to your laptop

06 / RETURN

Open a new laptop terminal. This creates a timestamped local folder, downloads both files, verifies the checksum and extracts results. It preserves earlier return folders.

LOCAL — laptop terminal, outside SSH

```
cd /home/bms/Desktop/BTP_Aegis-Perception
read -r -p "Jetson login (user@IP): " JETSON_TARGET
RETURN_DIR="artifacts/edge_v1/device_returns/$(date +%Y%m%d_%H%M%S)"
mkdir -p "$RETURN_DIR" &&
scp "$JETSON_TARGET:~/aegis-edge/aegis_jetson_results.tar.gz" \
  "$JETSON_TARGET:~/aegis-edge/aegis_jetson_results.tar.gz.sha256" \
  "$RETURN_DIR/" &&
(cd "$RETURN_DIR" &&
 sha256sum -c aegis_jetson_results.tar.gz.sha256 &&
 tar -xzf aegis_jetson_results.tar.gz)
printf 'Local results directory: %s\n' "$RETURN_DIR"
```

Expected: checksum OK and a results/ folder inside the printed directory. Keep the archive and checksum with the extracted evidence. If transfer or verification failed, resolve that before using the data.

Which files answer which questions?

Did the trials finish?
results/compression_exit_code.txt and results/*_compression/status.tsv

Was FP16 or INT8 faster?
results/*_compression/device_comparison.csv — compare throughput and median/p95 latency under identical conditions.

Did conversion retain quality? What precision actually ran?
results/engine_metadata/comparison_restormer_*.json — validation records and .layers.json details. The requested INT8 label does not prove every layer ran in INT8.

What happened to RAM, temperature and power?
results/*_compression/tegrastats_*.log, if generated. These are system readings, not automatically model-only memory or power.

Did restoration help compared with raw input?
results/paired_*/device_comparison.csv gives timings. Saved demos show examples. Neither establishes detector mAP without annotations.

Send the printed local directory or archive back for analysis. Do not choose INT8 based on FPS alone. A failed candidate is a valid experimental result and should stay in the report.

What this run does not finish

The four-image precision panel is an early quality check, not full validation. Larger per-weather PSNR/SSIM and annotated detection evaluation remain necessary for the selected precision. Repeat timing runs and assess longer thermal behaviour before claiming reliable deployment performance. Smoke is not fixed by conversion.

If you lose the terminal or hit an error

HELP / RECONNECT

Reconnect without restarting the comparison

LOCAL — new laptop terminal

```
read -r -p "Jetson login (user@IP): " JETSON_TARGET
ssh "$JETSON_TARGET"
```

JETSON — restore paths in the new shell

```
cd "$HOME/aegis-edge/aegis_restormer_edge_compression_20261024"
export AEGIS_PYTHON="$PWD/.venv/bin/python"
export REST_ENGINE=engines/comparison_restormer_fp16.engine
export DET_ENGINE=engines/comparison_yolov8n_fp16.engine
tail -n 40 results/compression_driver.log
```

Then use page 4's status check. If you previously selected FP32, change REST_ENGINE back to comparison_restormer_fp32.engine. Do not launch another comparison while one is running.

Common problems

SSH permission denied / timeout: check the Jetson account, address, password, network and SSH service with its owner. The DGX login is a different machine.

Missing venv or cv2: only for these reported missing OS packages, run the following on Jetson, then repeat page 3 setup. This requires the device administrator's sudo password.

JETSON — only if venv / OpenCV prerequisites are missing

```
sudo apt-get update &&
sudo apt-get install python3-venv python3-opencv
```

Missing CUDA/TensorRT or unsupported version: stop and obtain the correct JetPack setup from the device owner. This kit supports TensorRT 8.5/8.6 and 10.x, not 11.x. Do not blindly install desktop GPU wheels.

Out of memory / process killed: preserve logs, check free -h, close unrelated applications and return the evidence. The 512 MiB builder workspace is not a total RAM cap. A 136 MiB bundle does not establish that compilation fits a 4 GB device.

INT8 or FP16 rejected: read its .rejected.json and build log. Do not loosen quality limits just to pass. Use successfully validated FP16/FP32 for demos and retain failed trials.

Camera cannot open: check /dev/video*, index and permissions. Try the bundled image first. CSI cameras may need a different capture pipeline.

Output directory exists: use a fresh timestamped path; preserve old results. **No CSV:** inspect status.tsv and logs; there may be no completed benchmark.

Commands match the verified 44-file compression kit. Shell syntax is checked; actual Jetson execution remains pending. GNU nohup documentation: https://www.gnu.org/s/coreutils/manual/html_node/nohup-invocation.html